

에이전트에게 지식 부여,

RAG

에이전트에게 지식 부여, RAG

1. RAG 개념 이해하기
2. 파일 로드 및 청킹
3. 메타데이터로 데이터 구분하기
4. 임베딩 모델 이해하기
5. Chroma 벡터 저장소 만들고 저장하기
6. 벡터 저장소에서 검색하기
7. RAG 에이전트 만들기

1. RAG 개념 이해하기

- 회사의 사내 규정, 어제 발표된 최신 뉴스, 혹은 나만의 비공개 연구 노트 같은 정보는 LLM의 거대한 두뇌 속에 존재하지 않음
- 그래서 때로는 모르는 내용을 그럴듯하게 지어내는 환각(hallucination) 증세를 보이기도 함
- 에이전트가 필요할 때마다 도서관에서 책을 찾아보 듯, 방대한 문서 속에서 정확한 정보를 스스로 찾아낼 수 있다면 어떨까?
- 바로 이 ‘검색 능력(retrieval)’을 에이전트에게 장착하는 과정을 살펴봄
- 이것을 RAG(retrieval-augmented generation: 검색 증강 생성)라고 부름
- 이를 통해 LLM 모델이 가지고 있지 않은 지식들에 대한 답변을 해야 할 때 참고할 데이터베이스를 제공
- 이번 장에서는 텍스트를 숫자로 바꾸는 임베딩(embedding)과 이를 저장하는 벡터 저장소라는 새로운 개념을 다룸

1. RAG 개념 이해하기

● RAG(retrieval-augmented generation: 검색 증강 생성)

- 학습 기간 동안 공부한 내용은 잘 기억하지만, 그 이후에 일어난 최신 뉴스 등은 전혀 알지 못함
- LLM은 기본적으로 다음에 사용할 단어를 확률적으로 계산
- 문장의 참과 거짓을 구분할 수 없으므로 “모른다”라고 답변할 수는 없는 것
 - 이런 특성 때문에 없는 말을 지어내는 환각 증세를 보이게 됨
- 이 문제를 해결하기 위해 등장한 것이 RAG
- RAG는 LLM에게 ‘오픈 북 테스트’를 허용하는 것과 같음
- 질문을 받으면 바로 답하는 게 아니라, 주어진 문서를 찾아보고 그 내용을 바탕으로 답안을 작성하게 됨
- 학습하지 못한 내용이더라도, 문서를 검색할 수 있다면 충분히 유의미한 답변을 생성할 수 있게 됨

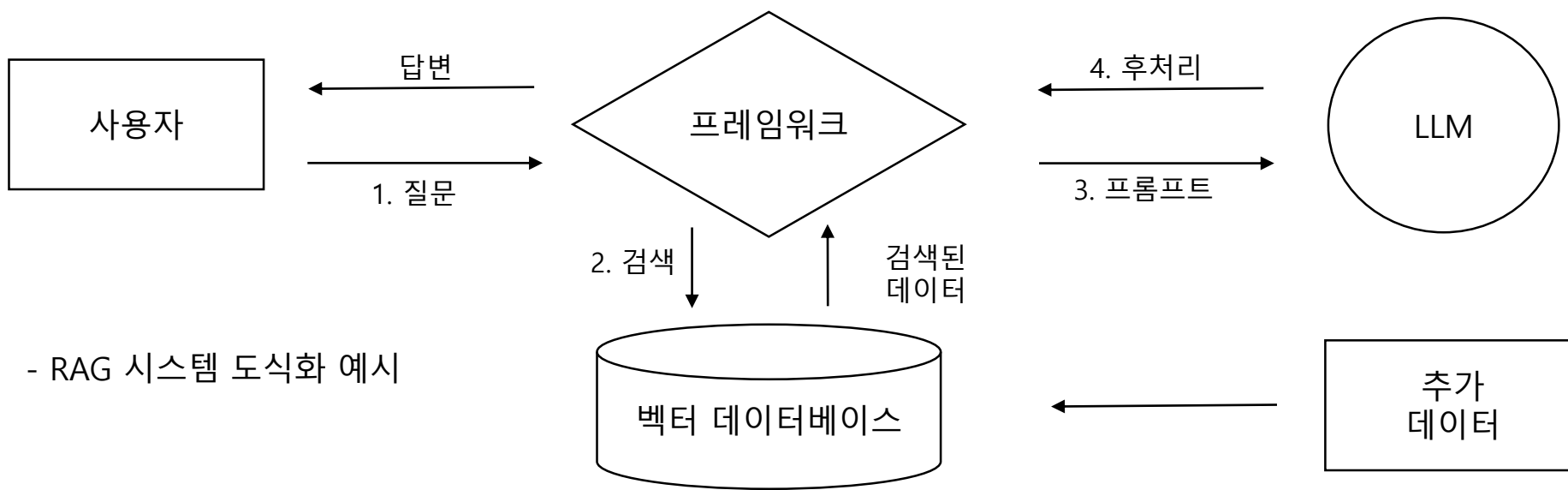
1. RAG 개념 이해하기

● RAG(retrieval-augmented generation: 검색 증강 생성)

• RAG의 동작 방식

- 1) 검색(retrieval): 사용자가 질문을 던지면, 벡터 저장소에서 질문과 가장 관련된 문서를 찾아냄
- 2) 증강(augmentation): 찾아낸 문서를 사용자 질문과 함께 묶어 프롬프트에 추가함. “이 문서를 참고해서 질문에 답해줘”라고 하는 것과 같음 (아래 그림의 3.프롬프트와 4.후처리 과정에서 진행되는 부분)
- 3) 생성(generation): LLM이 증강된 정보를 바탕으로 정확하고 검증된 답변을 생성함

- 이렇게 하면 LLM은 최신 정보를 반영할 수 있고, 근거 없는 거짓말을 할 위험도 획기적으로 줄어듦



1. RAG 개념 이해하기

● 벡터 임베딩(vector embedding)

- 매번 사용자의 질문이 발생할 때마다 모든 문서의 내용을 모델에게 읽도록 하는 것은 비효율적
- 유료 API를 사용하고 있는 입장에서 엄청나게 많은 비용이 발생할 수 있음
- 적절한 문서를 모델이 아닌 컴퓨터에게 찾도록 시킬 것
- AI도 아닌 일반적인 컴퓨터가 어떻게 문서들에서 ‘내 질문과 관련된 문서’를 찾아낼 수 있을까?
- 여기서 등장하는 핵심 기술이 **벡터 임베딩(vector embedding)**
- 컴퓨터는 텍스트의 의미를 인간처럼 직관적으로 이해하지 못함. 그래서 텍스트를 컴퓨터가 이해할 수 있는 언어, 즉 ‘숫자’로 바꿔 줘야 함
- 임베딩은 단어나 문장을 고정된 길이의 실수 리스트(벡터)로 변환하는 과정
- ‘의미가 비슷하면 벡터 간의 거리가 가깝다’는 원리를 이용해 키워드가 정확히 일치하지 않아도 의미상 관련된 문서를 찾아낼 수 있게 됨

1. RAG 개념 이해하기

● 벡터 저장소(vector store)

- 텍스트를 벡터로 변환했다면, 이 수많은 숫자 덩어리를 효율적으로 저장하고 검색할 수 있는 저장소
- ‘AI’나 ‘Artificial Intelligence’처럼 단어는 달라도 ‘의미(벡터)가 가까운’ 문서를 찾아낼 수 있음
- 대표적 벡터 DB중 Chroma(크로마)를 사용함
- Chroma는 별도의 복잡한 서버 구축없이 로컬 환경에서 파일 형태로 간편하게 사용할 수 있어 학습용이나 프로토타입 개발에 최적화되어 있음
- 또한, 속도가 빠르고 프로덕션 수준의 기능도 지원함

2. 파일 로드 및 청킹

- 이번 실습에서 에이전트에게 파이썬 기초 용어와 AI 용어를 정리한 파일과 펫보험 약관 두 가지 분야의 지식을 주입할 것임
- 이 두 가지 서로 다른 성격의 문서를 어떻게 불러올 수 있는지, AI가 이해하기 쉬운 크기로 자르는 방법은 무엇인지 알아봄

● 환경 설정

- 적당한 이름(여기서는 chap12)의 프로젝트 실습 폴더를 생성하고, 이곳에서 VS Code를 실행
- 실습 폴더에 data/ 폴더를 만들고, 약관 pdf와 용어 txt 파일을 담음
- 가상 환경을 생성, 활성화하여 실습에 필요한 패키지 목록들을 설치
- 가상 환경 설정

```
# 가상 환경 생성
$ python -m venv .venv

# 윈도우 가상 환경 활성화
$ source .venv/Scripts/activate
```


2. 파일 로드 및 청킹

- 환경 설정

- 필요 라이브러리 설치

```
(.venv)
```

```
$ pip install langchain langchain-openai langchain-community pypdf langchain_text_splitters langchain-chroma
```

```
# 의존성 패키지 목록 관리
```

```
(.venv)
```

```
$ pip freeze > requirements.txt
```

2. 파일 로드 및 청킹

● 파일 로더로 문서 불러오기

- document_chunking.py 파일 생성
- 데이터 불러오기

```
from langchain_community.document_loaders import TextLoader, PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. AI 용어 로드 (텍스트 파일)
ai_loader = TextLoader("data/Python_AI_Glossary_Guide.txt", encoding="utf-8")
ai_docs = ai_loader.load()

# 2. 펫보험 약관 로드 (PDF 파일)
insurance_loader = PyPDFLoader("data/meritz_pet_insurance.pdf")
insurance_docs = insurance_loader.load()
```

- 파일을 열고, 임베딩 작업을 하기 위한 데이터를 필요로 하기 때문에 조금 특별한 클래스 사용
- TextLoader와 PyPDFLoader 클래스를 불러와 읽고자 하는 파일의 경로를 첫 번째 인자로 작성
- TextLoader의 경우, 한글 데이터를 올바르게 불러오기 위해 encoding 인자도 작성해 줌
- 이렇게 생성한 각 객체에서 load 메서드를 사용하여 문서 정보를 불러올 수 있음

2. 파일 로드 및 청킹

● 파일 로더로 문서 불러오기

• 데이터 출력하기

```
print(f"AI 용어: {len(ai_docs)}개 문서 로드")
print(f"메타데이터: {ai_docs[0].metadata}")
print(f"내용: {ai_docs[0].page_content[:50]}...")
print()
print(f"펫보험 약관: {len(insurance_docs)}개 페이지 로드")
print(f"메타데이터: {insurance_docs[0].metadata}")
print(f"내용: {insurance_docs[0].page_content[:50]}...")
```

- 두 객체는 모두 동일한 속성을 통해 정보를 불러올 수 있음
- 예시에서는 첫 번째 문서 혹은 페이지의 내용만 확인
- 출력문을 작성하고 터미널에서 python document_chunking.py 명령어로 코드 실행
- 펫보험 PDF 파일의 경우, 여러 페이지의 문서를 불러올 수 있으니 다른 페이지의 내용들도 확인
- 두 클래스는 파일을 불러오기만 하는 파이썬 open 함수와 달리 데이터의 출처, 벡터 임베딩을 위한 처리, 텍스트 분할을 위한 메서드 등을 제공함.

```
AI 용어: 1개 문서 로드
메타데이터: {'source': 'data/Python_AI_Glossary_Guide.txt'}
내용:
파이썬 기초 개념
---
```

```
펫보험 약관: 28개 페이지 로드
메타데이터: {'producer': 'iLovePDF', 'creator': 'PyPDF', 'creationdate': '', 'moddate': '2024-10-11T00:26:06+00:00', 'source': 'data/meritz_pet_insurance.pdf', 'total_pages': 28, 'page': 0, 'page_label': '1'}
내용: 장기 약관_제2024-8호반려동물 실손의료비보험(무)펫퍼민트 Puppy&Family보험24...
```

2. 파일 로드 및 청킹

● 문서 청킹하기

- 문서를 불러왔지만, 이대로는 바로 사용할 수 없음. 예를 들어, 수백 페이지짜리 펫보험 약관 전체를 한 번에 LLM에게 던져주고 “여기서 찾아봐”라고 하면, LLM이 한 번에 읽을 수 있는 양(토큰)을 넘어설 가능성이 높음
- 또한, 너무 많은 정보 속에서 핵심을 놓치기 쉬움
- 문서를 의미 있는 작은 조각으로 나누는 작업을 하는 데, 이를 **청킹(chunking)**이라고 함
- 단순히 ‘100 글자씩 자르기’가 아니라 ‘겹치게 자르기(overlap)’를 해야 한다는 점

[문서 원본]

인공지능은 인간의 지능을 모방하여 작업을 수행하고...(생략)

[잘못된 청킹 (문맥 끊김)]

청크 1: ...인간의 지능을 모방하여

청크 2: 작업을 수행하고...

[올바른 청킹 (Overlap)]

청크 1: ...인간의 지능을 모방하여 작업을

청크 2: 모방하여 작업을 수행하고...

- 겹치는 구간을 두면 문장이 중간에 잘려도 의미를 잃어버릴 가능성이 줄어 듦

2. 파일 로드 및 청킹

● 문서 청킹하기

- 랭체인의 langchain_text_splitters에서 제공하는 RecursiveCharacterTextSplitter를 사용하여 실제로 문서를 잘라보겠음
- RecursiveCharacterTextSplitter는 재귀적으로 정해진 우선순위(단락, 줄바꿈, 공백/단어, 글자 단위)에 따라 분할을 진행해 줌
- 우선은 200자 단위로 자르되, 앞뒤 내용을 20자씩 겹치게 설정
- 이전 작성 출력문(print)은 잠시 주석 처리하고, 그 아래에 이어서 코드 작성

```
# 청킹 설정: 200자씩 자르고, 20자는 겹치게(overlap) 설정
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,      # 청크 크기
    chunk_overlap=20,    # 청크 겹침
)

# AI 용어 청킹
ai_chunks = text_splitter.split_documents(ai_docs)

# 펫보험 청킹
insurance_chunks = text_splitter.split_documents(insurance_docs)

# 청킹 결과 출력
print(f"AI 용어 청킹: {len(ai_chunks)}개 청크로 분할")
print(ai_chunks[0].page_content)
print()
print(f"펫보험 청킹: {len(insurance_chunks)}개 청크로 분할")
print(insurance_chunks[0].page_content)
```

2. 파일 로드 및 청킹

● 문서 청킹하기

• 출력 결과

AI 용어 청킹: 100개 청크로 분할
파이썬 기초 개념

OOP (Object-Oriented Programming)

정의: 객체 지향 프로그래밍(OOP)은 데이터와 그 데이터에 관련된 함수(메서드)를 하나의 "객체"로 묶어 표현하는 방식입니다.

설명: Python은 OOP를 지원하며, 클래스와 객체를 사용해 코드를 재사용성 높이고 유지보수를 용이하게 만듭니다.

펫보험 청킹: 142개 청크로 분할

장기 약관_제2024-8호반려동물 실손의료비보험(무)펫퍼민트 Puppy&Family보험2404보험약관

판매버전 1.0판매개시 2024. 4. 1※본 약관은 관계 법령 및 내부통제기준에 따른 절차를 거쳐 제공됩니다.

- 청크 처리된 문서에서 원하는 정보를 출력하는 방법은 이전과 동일한 속성으로 가능
- 청크 처리된 AI 용어와 펫보험 문서의 내용을 출력해 보면, 단순히 정해진 범위(200자)에서 무작정 잘라내는 것과는 다르다는 것이 느껴지나요?
- 이번에는 각각 100개, 142개의 청크로 분할되었다고 하니, 인덱스를 변경해 보며 각각 어떤 내용들이 담겨 있는지 확인
- chunk_size=200을 설정했지만, 이 숫자는 문서의 성격에 따라 끊임없이 실험해야 하는 ‘하이퍼파라미터(hyperparameter)’임
- 청크 사이즈를 결정하는 것은 ‘질문과의 높은 연관성(specificity)’과 ‘답변에 필요한 충분한 맥락(context preservation)’ 사이의 균형을 맞추는 일이기 때문
- 청크가 너무 작으면 답변에 필요한 핵심이 흩어져 버려 LLM이 올바르게 답변하지 못할 위험이 있고, 반대로 너무 크면 LLM이 질문에 답하기 위해 불필요한 정보를 너무 많이 읽게 되어 오히려 정답을 놓치거나, 환각을 생성할 여지가 생김
- ‘기대하는 답변의 근거가 되는 내용 전체가 단일 청크에 담길 수 있는 최소한의 범위’가 되도록 적절한 크기를 찾는 것이 핵심

3. 메타데이터로 데이터 구분하기

- 앞서 청킹 작업 후 이 조각들을 이대로 섞어두면 나중에 문제가 생길 수 있음
- 단순히 텍스트 내용만 저장하는 것이 아니라, 이 내용이 ‘어디서 왔는지’, ‘어떤 종류인지’를 알려주는 메타데이터(metadata)를 함께 저장해야, ‘파이썬 함수가 뭐야?’라고 물었는데, 펫보험 약관에서 답을 찾는 오류를 줄일 수 있음
- 메타데이터는 데이터에 대한 ‘추가 정보’를 의미
- 이를 이용해 나중에 에이전트가 “펫보험 관련 질문이니 펫보험 문서에서만 찾아봐”라고 명령할 수 있게 만들 것임

3. 메타데이터로 데이터 구분하기

● 메타데이터 추가

- 파이썬의 딕셔너리 형태인 metadata 속성에 새로운 키와 값을 추가하기만 하면 됨
- 반복문을 사용해 각 청크들의 source_type과 category 속성에 동일한 값들을 할당

```
# 1. AI 용어 청크에 메타데이터 추가
for chunk in ai_chunks:
    chunk.metadata["source_type"] = "glossary"
    chunk.metadata["category"] = "AI & Python"

# 2. 펫보험 청크에 메타데이터 추가
for chunk in insurance_chunks:
    chunk.metadata["source_type"] = "insurance"
    chunk.metadata["category"] = "Pet Insurance"

print(f"{ai_chunks[0].metadata}")
```

```
{'source': 'data/Python_AI_Glossary_Guide.txt', 'source_type': 'glossary', 'category': 'AI & Python'}
```

- 랭체인이 기본적으로 만들어 준 정보(source) 외에 우리가 정의한 source_type과 category가 잘 들어간 것을 확인
- 태그 정보들을 토대로 검색의 품질을 향상시킬 수 있게 됨
- 예를 들어, 주제 상관없이 가장 유사한 내용을 찾고 싶을 때는 태그와 상관없이 검색하도록 지시하면 됨
- 전문적인 에이전트 만들고자 할 때는 사용자의 의도에 따라 검색 범위를 좁혀주는 것이 정확도(accuracy)를 높이는 확실한 방법

4. 임베딩 모델 이해하기

● 임베딩 모델이란?

- 텍스트를 숫자로 바꾼다는 것은, 사실 텍스트의 의미를 다차원 공간의 좌표로 표현한다는 뜻. 이 좌표를 벡터(vector)라고 부름
- 임베딩 모델(embedding model) 역할
 - 문장이 가진 미묘한 의미, 문맥, 뉘앙스를 파악
 - 이 의미를 가장 잘 나타낼 수 있는 수백(또는 수천)개의 차원으로 이루어진 공간에 고유의 좌표를 부여함

텍스트	임베딩 모델의 역할	벡터 공간에서의 결과
강아지 보험	금융 상품과 반려동물의 의미를 파악	V1 좌표(0.8, -0.3, 0.1, ...)
반려견 보험	동義어로 인식하고 유사한 의미를 파악	V2 좌표(0.7, -0.2, 0.1, ...)(V1과 매우 가까움)
자동차 보험	보험은 같지만 대상(자동차)이 다름을 파악	V3 좌표(-0.5, 0.9, -0.4, ...)(V1과 멀다)

- 예를 들어, ‘강아지 보험’이라는 단어와 ‘자동차 보험’이라는 단어의 벡터는 ‘보험’ 관련 단어들이 모여 있는 공간에 가깝게 위치하게 되지만, 대상은 ‘동물’과 ‘자동차’로 다르기 때문에 상대적으로 먼 거리에 위치하게 됨
- 임베딩 모델은 글자가 아닌 의미적 유사성을 기준으로 벡터를 생성하며, 의미가 비슷하면 벡터간의 거리가 가깝다는 원리를 이용해 우리는 키워드가 정확히 일치하지 않아도 의미상 관련된 문서를 찾아낼 수 있게 됨
- 이 원리 덕분에, 우리가 “시란 무엇인가요?”라고 질문을 던지면, 임베딩 모델은 이 질문을 벡터로 변환한 후, 이 벡터와 가장 가까운 거리에 있는 답변 근거(청크)를 벡터 저장소에서 찾아낼 수 있는 것임

4. 임베딩 모델 이해하기

- 오픈AI 임베딩 모델 설정하기

- (실습)에서는 오픈AI의 **'text-embedding-3-small'** 모델 사용
- 환경 설정을 위해 **'env'** 파일을 생성하고 키를 작성하여 관리
 - **.env** 파일은 환경 변수들을 키-값 쌍으로 저장하여 관리
- **'create_vector_db.py'** 파일을 생성하여 이곳에서 임베딩 모델을 설정
- 이후, 벡터 저장소를 생성하는 과정도 이 파일에서 진행

4. 임베딩 모델 이해하기

- 오픈AI 임베딩 모델 설정하기

- 임베딩 모델 설정하기

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from document_chunking import ai_chunks, insurance_chunks

# .env 파일에서 환경 변수를 로드합니다.
load_dotenv()

# 임베딩 모델을 설정합니다.
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

- 임베딩을 위한 모델은 오픈AI의 OpenAIEmbeddings을 사용
 - 복잡한 수학 공식 없이 랭체인이 모든 것을 처리해 주기 때문에, 우리는 모델만 선택하면 됨

5. Chroma 벡터 저장소 만들고 저장하기

- 4절에서 임베딩 모델이라는 ‘번역가’가 준비되었고, 2/3절에서 잘게 쪼개고 꼬리표를 단 지식 조각들이 준비
- 이제 이 모든 것을 벡터 저장소(Vector Store)라는 영구적인 저장소에 보관할 차례

● 왜 Chroma를 선택했는가?

- Chroma는 벡터 저장소의 한 종류
- 일반적인 SQL 데이터베이스(MySQL, PostgreSQL)가 텍스트나 숫자를 저장한다면, Chroma는 벡터를 효율적으로 저장하고 검색하도록 설계된 전문 데이터베이스
- 일반 데이터베이스는 ‘완벽한 일치’를 기준으로 검색
 - `SELECT * FROM documents WHERE title = ‘펫보험’`
- 벡터 저장소는 ‘의미적 유사성(semantic similarity)’을 기준으로 검색
- 벡터 저장소 종류
 1. Chroma: 상대적으 가볍고, 설정이 간단하며, 랭체인과의 통합이 잘 되어 있음
 2. Pinecone: 클라우드 기반, 대규모 운영에 유리하지만 유료로 운영됨
 3. Weaviate: 다양한 기능을 제공하지만 그만큼 설정이 복잡함
 4. Milvus: 오픈 소스지만, 별도 인프라가 필요함

5. Chroma 벡터 저장소 만들고 저장하기

- Chroma 벡터 DB 생성 및 저장

- 벡터 저장소 생성을 위한 준비
 - 2절에서 작성한 document_chunking 모듈에서 청크 데이터를 불러옴
 - 데이터베이스를 어디에 생성할 것인지 위치를 지정
 - 이 코드는 임베딩 모델을 불러왔었던 'create_vector_db.py'에 이어서 작성함

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from document_chunking import ai_chunks, insurance_chunks

... (생략) ...

# 1. 모든 청크를 하나의 리스트로 합칩니다.
# (8.2절에서 생성된 ai_chunks와 insurance_chunks를 사용합니다.)
all_chunks = ai_chunks + insurance_chunks

# 2. Chroma DB를 생성하고 영구적으로 저장할 경로를 지정합니다.
persist_directory = "chroma_data"

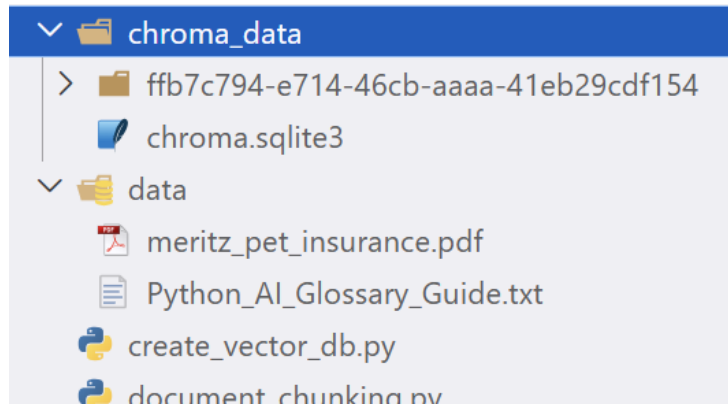
# 3. Chroma DB 생성 및 저장 (실행 시간이 다소 걸릴 수 있습니다.)
vectorstore = Chroma.from_documents(
    documents=all_chunks,
    embedding=embedding_model,
    persist_directory=persist_directory
)
```

5. Chroma 벡터 저장소 만들고 저장하기

- Chroma 벡터 DB 생성 및 저장

- 벡터 저장소 생성을 위한 준비

- 'create_vector_db.py'를 실행하면, 지정한 위치("**chroma_data**")에 데이터베이스 생성



- 주의할 점

- 만약, 코드를 수정하고 데이터베이스를 다시 생성하고 싶다면, 반드시 기존의 데이터베이스를 삭제하고 진행해야 함
 - 삭제하지 않고 이 코드를 반복해서 실행하면, 같은 내용이 계속해서 데이터베이스에 추가되어 중복 데이터가 쌓이게 됨

6. 벡터 저장소에서 검색하기

- 에이전트가 이 저장소를 실제로 활용하여 질문에 대한 근거를 찾는 방법을 알아볼 차례
- 이 과정은 RAG의 첫 번째이자 가장 중요한 단계인 검색(retrieval)을 담당하며, 여기서 찾아낸 문서의 품질이 최종 답변의 품질을 결정
- 좋은 RAG 시스템 = 좋은 검색 엔진

6. 벡터 저장소에서 검색하기

- 저장된 벡터 저장소 불러오기와 Retriever 객체 생성

- RAG 시스템의 검색기(Retriever)는 저장된 벡터 저장소에서 사용자의 질문과 가장 유사한 문서들을 찾아와서 반환하는 객체
- 여기서 ‘검색’이란, 사용자의 질문을 벡터로 변환한 후, 저장된 모든 문서 벡터와의 거리를 계산해서 가장 유사한 문서들을 찾아내는 작업 의미
- 검색을 수행하기 위해 벡터 저장소 객체(retrieved_vectorstore)를 불러와 as_retriever 메서드를 적용하여 검색기 객체를 만듦
- 이 객체는 LLM과 벡터 저장소 사이의 ‘검색 중개인’ 역할을 수행

6. 벡터 저장소에서 검색하기

- 저장된 벡터 저장소 불러오기와 Retriever 객체 생성
 - 벡터 저장소 불러오기와 검색기 객체 생성
 - 중복 벡터 저장소 생성을 막기 위해 새로운 파일 'retriever_search.py' 파일을 생성해서 진행
 - 반드시, 처음 벡터 저장소를 만들 때와 동일한 임베딩 모델을 사용해야 함

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

# .env 파일에서 환경 변수를 로드합니다.
load_dotenv()
# 임베딩 모델을 설정합니다.
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")

# 1. 저장된 벡터 데이터베이스를 다시 불러옵니다. (생성이 아님!)
retrieved_vectorstore = Chroma(
    persist_directory="chroma_data",
    embedding_function=embedding_model # 4절에서 설정한 모델 사용
)

# 2. 로드가 성공했는지 검증합니다.
collection = retrieved_vectorstore.get()
print(f"총 {len(collection['ids'])}개 문서 확인.")

# 3.2 '검색(Retriever)' 객체를 생성합니다.
retriever = retrieved_vectorstore.as_retriever()
```

6. 벡터 저장소에서 검색하기

- 유사도 검색과 Top-k의 중요성

- 유사도 검색(similarity search)

- 사용자 질문 벡터와 데이터베이스 내 모든 문서 벡터 간의 거리를 계산하여, 가장 가까운 문서들(즉, 의미적으로 가장 유사한 문서들)을 순서대로 반환하는 방식

- Top-k 값

- k는 LLM에게 근거 자료로 제공할 ‘가장 유사한 문서의 개수’ 의미. 기본값은 보통 ‘4’
 - k 값을 너무 작게 설정한다면 질문에 답하기 위한 문맥이 여러 청크에 분산되어 있어, 핵심 근거가 누락될 수 있음
 - 즉, 정보 누락으로 인한 답변 실패 위험성이 증가
 - k 값을 너무 크게 설정한다면 불필요한 노이즈 정보가 LLM에게 전달되어 답변의 정확도가 떨어지고, LLM의 토큰 제한에 걸리거나 비용이 증가
 - 즉, 맥락 혼란으로 인한 환각 증세가 발생할 위험 증가
 - 앞선 청크 사이즈의 설명과 유사
 - k 값 역시 하이퍼파라미터로 ‘답변에 필요한 모든 정보가 포함되면서도 불필요한 정보는 최소화되는 지점’을 찾아야 함

6. 벡터 저장소에서 검색하기

- 유사도 검색과 Top-k의 중요성

- AI 용어에 대한 일반 검색(k=4)
 - 지금은 일단 기본값으로 설정하고 검색이 정상적으로 동작하는지 부터 확인
 - 검증을 위해 우선 retriever 객체를 통해 얻고자 하는 정보를 질문. 이때 또 다시 invoke 메서드가 사용됨
 - 'retriever_search.py' 파일에 이어서 작성

```
# 4. 검색어를 사용하여 문서를 검색합니다.
query = "파이썬 반복문에 대해 알려줘"
docs = retriever.invoke(query)

print(f"검색 결과 문서 수: {len(docs)}개")
for i in range(len(docs)):
    print(f"\n--- 문서 {i+1} ---")
    print(docs[i].page_content)
```

6. 벡터 저장소에서 검색하기

- 유사도 검색과 Top-k의 중요성

- 출력 결과 확인

검색 결과 문서 수: 4개

--- 문서 1 ---

파이썬 기초 개념

OOP (Object-Oriented Programming)

... (생략) ...

--- 문서 2 ---

설명: 주로 간단한 기능을 빠르게 정의할 때 사용됩니다.

List Comprehension (리스트 컴프리헨션)

... (생략) ...

Dictionary Comprehension (딕셔너리 컴프리헨션)

--- 문서 3 ---

Iterable (이터러블)

... (생략) ...

GIL (Global Interpreter Lock)

--- 문서 4 ---

예시: k-폴드 교차 검증에서는 데이터를 k개의 폴드로 나누어, 각 폴드가 검증 데이터가 되도록 ... (생략) ...

하이퍼파라미터 (Hyperparameter)

- 파이썬 반복문에 대한 질문의 검색 결과로 4개의 문서가 반환됨
- 대부분 만족스럽게 검색

6. 벡터 저장소에서 검색하기

● 메타데이터 필터로 검색하기

- 3절에서 문서에 `source_type` 이라는 꼬리표를 달았던 가장 큰 이유는 바로 필터링(filtering) 검색을 통해 검색의 전문성을 높이기 위함
- 우리가 만든 데이터베이스에는 AI 용어와 펫보험 약관이 섞여 있음
- 사용자가 ‘파이썬 반복문’을 물었을 때, 펫보험에 대한 설명이 검색 결과에 섞여 들어간다면 LLM이 헛갈리거나 불필요한 내용을 함께 읽게 됨. 이때 필터링을 활용
- 필터링은 벡터 검색(유사도 계산)을 수행하기 이전에 특정 조건을 만족하는 문서만 추려내는 사전 작업
- 이는 검색의 속도와 정확도를 동시에 높여 줌

6. 벡터 저장소에서 검색하기

- 메타데이터 필터로 검색하기

- AI 용어 문서 내에서만 검색하기
 - 이전 작성한 코드를 수정하여 필터링 검색 기능 구현
 - 우선 필터링 조건을 딕셔너리 형태로 만들
 - 더 다양한 조건을 작성할 수도 있지만, 지금은 문서 종류가 2개 밖에 없으므로 source_type만 지정해도 충분히 효과적인 필터링이 가능
 - 이후, as_retriever 메서드로 retriever 객체를 생성할 때 search_kwargs 인수에 filter 조건을 딕셔너리 형태로 전달

```
# 3.1 필터 조건 설정합니다. (새롭게 추가된 부분)
ai_fliter = {
    'source_type': 'glossary',
    #'source_type': 'insurance',
}

# 3.2 '검색(Retriever)' 객체를 생성합니다.
retriever = retrieved_vectorstore.as_retriever(
    search_kwargs={
        'filter': ai_fliter
    },
```

- 출력 결과 확인

6. 벡터 저장소에서 검색하기

- MMR 검색으로 답변의 다양성 확보

- 유사도 검색은 가장 가까운 벡터만 가져오기 때문에 문제가 발생
- 예를 들어, 질문과 아주 유사한 내용이 3개의 연속된 청크에 걸쳐 있을 경우, 유사도 검색은 이 3개의 청크만 반환함
- 이들은 내용이 거의 중복되기 때문에 LLM에게 중복된 근거만 제공하는 결과를 냄
- 이러한 검색 결과의 중복성 문제를 해결하기 위해 MMR(Max Marginal Relevance) 검색을 사용할 수 있음
- MMR을 사용하면, 가장 유사한 문서 하나를 찾은 다음, 다음 문서를 찾을 때는 질문과도 유사하고, 이전에 찾은 문서와는 거리가 먼 문서를 선택하여 다양하고 풍부한 맥락을 LLM에게 제공할 수 있음

6. 벡터 저장소에서 검색하기

● MMR 검색으로 답변의 다양성 확보

- MMR 검색 적용
 - 검색 방식을 변경하는 방법 역시, 필터링을 적용할 때와 마찬가지로 retriever 객체를 생성할 때, 검색 방식을 지정해 주기만 하면 됨
 - 이전 검색 방식과 검색 결과를 비교하기 위해서 새로운 retriever 객체를 생성
 - search_type으로 검색 방식을 제어하고, search_kwargs으로 다양한 하이퍼파라미터를 정의
 - 이번에는 k와 fetch_k도 추가
 - fetch_k : 먼저 찾아올 후보군 문서의 개수. MMR 검색 방법을 생각했을 때, 다양한 후보군이 있으면 더욱 풍부한 맥락을 제공

```
# 5. 검색 타입을 'mmr'로 변경하여 새로운 retriever를 생성합니다.
```

```
mmr_retriever = retrieved_vectorstore.as_retriever(  
    search_type="mmr",  
    search_kwargs={  
        "k": 5,  
        "fetch_k": 20,  
        'filter': ai_fliter  
    }  
)
```

```
# 6. MMR 검색 수행
```

```
mmr_query = "파이썬 반복문에 대해 알려줘"
```

```
# mmr_query = "보험 약관 정리해 줘"
```

```
mmr_docs = mmr_retriever.invoke(mmr_query)
```

```
print(f"MMR 검색 결과 문서 수: {len(mmr_docs)}개")
```

```
for i in range(len(mmr_docs)):
```

```
    print(f"\n--- MMR 문서 {i+1} ---")
```

```
    print(mmr_docs[i].page_content)
```


6. 벡터 저장소에서 검색하기

- MMR 검색으로 답변의 다양성 확보

- 출력 결과 확인

MMR 검색 결과 문서 수: 5개

--- MMR 문서 1 ---

파이썬 기초 개념

OOP (Object-Oriented Programming)

... (생략) ...

--- MMR 문서 2 ---

Iterable (이터러블)

... (생략) ...

--- MMR 문서 3 ---

예시: 미로 탈출 문제에서 에이전트는 각 위치(상태)에서 이동 방향(행동)에 따른 보상을 학습하여 최단 경로를 찾습니다. Q-러닝을 통해 에이전트는 반복적인 시도를 통해 최적의 경로를 학습합니다.

... (생략) ...

--- MMR 문서 4 ---

FastAPI

정의: FastAPI는 Python의 비동기 웹 프레임워크로, 빠르고 효율적인 API를 구축하는 데 사용됩니다.

... (생략) ...

--- MMR 문서 5 ---

정의: 교차 검증은 모델의 일반화 성능을 평가하기 위해 데이터를 여러 부분으로 나누어 반복적으로 학습하고 검증하는 방법입니다. 이를 통해 모델의 과적합을 방지하고 성능을 신뢰성 있게 측정할 수 있습니다.

6. 벡터 저장소에서 검색하기

- MMR 검색으로 답변의 다양성 확보

- “보험 약관을 정리해 주세요”라는 질문에 대한 유사도 검색 방식의 출력 결과 확인

검색 결과 문서 수: 4개

--- 문서 1 ---

4약관 내용 중 어려운 보험용어는 용어해설, 약관 본문 Box안 예시 등을 참고하시면 약관 이해에 도움이 됩니다.➡용어 해설 P43

5스마트폰으로 QR코드를 인식하면 약관해설 동영상, 보험금 지급절차, 전국 지점 등을 쉽게 안내받을 수 있습니다.➡QR코드 P13

--- 문서 2 ---

4약관 의핵 심체 크항목 쉽게 찾기(보 통 약관 기 준)보험약관 핵심사항 등과 관련된 해당 조문, 쪽수 및 영상자료 등을 안내드리오니, 보험회사로부터 약관을 수령한 후, 해당 내용을 반드시 확인·숙지하시기 바랍니다.

1보험금지급및지급제한사항제3조(보험금의 지급사유) P50

--- 문서 3 ---

1보험 약관이 란?보험약관은 가입하신 보험계약의 내용 및 조건 등을 미리 정하여 놓은 계약조항으로 보험계약자와 보험회사의 권리 및 의무를 규정하고 있습니다. 특히, 청약철회, 계약취소, 보험금 지급 및 지급제한 사항 등 보험계약의 중요사항에 대한 설명이 들어 있 으니 반드시 확인하셔야 합니다.2한눈 에보 는약관 의구 성

--- 문서 4 ---

7약관조항 등이 음영·컬러화 되거나 진하게 된 경우 보험금 지급 등 약관 주요 내용이므로 주의 깊게 읽기 바랍니다.6기타문의사항※ 기타 문의사항은 당사 홈페이지(www.meritzfire.com), 고객센터(1566-7711)로 문의 가능※ 보험상품 거래단계별 필요한 금융꿀팁 또는 핵심정보 등은 금융감독원 금융소비자정보

6. 벡터 저장소에서 검색하기

- MMR 검색으로 답변의 다양성 확보

- “보험 약관을 정리해 주세요”라는 질문에 대한 MMR 검색 방식의 출력 결과 확인

MMR 검색 결과 문서 수: 5개

--- MMR 문서 1 ---

4약관 내용 중 어려운 보험용어는 용어해설, 약관 본문 Box안 예시 등을 참고하시면 약관 이해에 도움이 됩니다. ➡용어 해설 P43
5스마트폰으로 QR코드를 인식하면 약관해설 동영상, 보험금 지급절차, 전국 지점 등을 쉽게 안내받을 수 있습니다. ➡QR코드 P13

--- MMR 문서 2 ---

!주의소액 보험금 청구시 진단서 제출이 면제되는 등 청구서류가 간소화되는 경우도 있으니, 보험금청구 전에 보험회사에 제출서류를 확인하시기 바랍니다. 보험금 청구서류구분진단서입·퇴원확인서수술확인서통원확인서기타공통사망●(사망진단서) 영상자료 등
청구서신분증개인정보동의서

--- MMR 문서 3 ---

보통약관제15조제17조보험계약자 또는 피보험자는 보험계약을 청약할 때 청약서의 질문사항(중요사항)에 대하여 사실대로 알려야 합니다. 계약 해지 가능(회사)보장 제한 가능(회사)계약자또는피보험자고의 또는 중과실로사실과 다르게 알린 경우!주의① 청약서의 질문사항에 대하여 보험설계사에게만 구두로 알렸을 경우 보험계약前 알릴의무를 이행한 것으로 볼 수 없습니다.②

--- MMR 문서 4 ---

장기 약관_제2024-8호반려동물 실손의료비보험(무)펫퍼민트 Puppy&Family보험2404보험약관
판매버전 1.0판매개시 2024. 4. 1※본 약관은 관계 법령 및 내부통제기준에 따른 절차를 거쳐 제공됩니다.

--- MMR 문서 5 ---

5. 소비자가 반드시 알아두어야 할 상품의 주요 특성 ❶ 보장성보험

6. 벡터 저장소에서 검색하기

- MMR 검색으로 답변의 다양성 확보

- 유사도 검색 방식과 MMR 검색 방식의 결과 비교
 - MMR 검색 방식의 결과가 더 다양한 주제의 문서를 찾아오는 것을 볼 수 있음
 - 유사도 검색은 조항만을 나열한 반면, MMR 검색은 약관, 주의사항, 체결한 보험 정보 등 여러 맥락의 정보를 제공
 - 이 문서들이 에이전트에게 전달되면 LLM은 문서를 토대로 더 높은 수준의 답변을 생성할 수 있게 됨
- 검색 방법을 바꾸는 것만으로도 RAG 시스템의 답변 품질을 향상시킬 수 있음
- 더 높은 품질을 목표로 한다면, 이번 장에서 다룬 전반적인 부분(칭킹, 임베딩 방식, 각종 하이퍼파라미터 등)을 여러분이 가지고 있는 데이터에 맞게 조절해야 됨

7. RAG 에이전트 만들기

- 에이전트의 두뇌와 RAG의 지식을 하나로 합쳐 지능형 에이전트를 완성해 볼 차례
- 이번 절의 핵심은 RAG 검색 기능을 에이전트가 활용할 수 있도록 retriever 객체를 랭체인 도구 형태로 변환하는 과정

● 검색 Tool 만들기

- 에이전트는 파이썬 함수에 tool 데코레이터 붙인 형태나, 랭체인에서 제공하는 Tool 객체만 인식할 수 있음
- 6절에서 만든 retriever 객체는 바로 사용할 수 없으므로, 이를 에이전트가 이해할 수 있는 형태의 도구로 변환해야 함

7. RAG 에이전트 만들기

- 검색 Tool 만들기

- 파일 생성과 기본 설정

- 'RAG_agent.py' 파일을 생성하고, 벡터 저장소를 불러옴

1. '.env' 파일에서 오픈AI 키 불러오기
2. 임베딩 모델 정의하기
3. Chroma 벡터 저장소 불러오기

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

load_dotenv()
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")

vector_store = Chroma(
    persist_directory="chroma_data",
    embedding_function=embedding_model
)
```

7. RAG 에이전트 만들기

● 검색 Tool 만들기

- Pydantic으로 도구 입력 구조 정의
 - 현재 벡터데이터베이스에는 2종류의 문서가 있음
 - 사용자의 질문이 무엇인지에 따라 서로 다른 필터링의 적용되어야 함
 - 이를 pydantic으로 어떻게 필터링할 것인지 구조화하고, 키워드를 선정하는 것은 에이전트에게 맡기도록 함
 - retriever 객체로 검색하기 위해 필요한 사용자 질문, 필터링할 메타데이터 정보, k 값 필드를 정의
 - 필터링을 위한 source_type의 경우, 확장성을 고려하여 all 값도 추가

```
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="검색할 질문이나 키워드")
    source_type: str = Field(
        default="all",
        description="검색 대상: 'glossary' (AI/파이썬), 'insurance' (펫보험), 'all' (전체)"
    )
    k: int = Field(default=3, description="반환할 결과 개수")
```

7. RAG 에이전트 만들기

● 검색 Tool 만들기

• search_docs 도구 구현

- retriever 객체를 정의하고, 도구화하는 작업 진행

• tool 데코레이터 정의

- 정확한 데이터 입력을 위해 args_schema 인자로 입력 데이터의 구조를 SearchInput 모델로 지정
- 에이전트가 어떤 도구인지 이해할 수 있도록 간단한 설명을 작성(docstring과 용도가 다름을 기억)
- search_docs 함수의 기본 로직 작성
 - search_docs의 용도는 에이전트가 source_type을 결정하면, 그 값에 따라 필터링된 문서를 검색하는 것
 - 이를 위해, 적절한 retriever 객체가 생성될 수 있도록 조건문을 작성
 - 상황에 따라, 필터가 필요 없을 때는 None을 전달하는 것으로 필터를 생략할 수도 있음
 - as_retriever 메서드로 retriever 객체를 생성하고, invoke 메서드로 사용자 질문에 맞는 문서를 검색하도록 함

7. RAG 에이전트 만들기

● 검색 Tool 만들기

- search_docs 도구 구현

```
from langchain.tools import tool

@tool(args_schema=SearchInput, description="RAG 지식베이스에서 관련 문서를 검색합니다.")
def search_docs(query, source_type="all", k=3):
    """RAG 지식베이스에서 관련 문서를 검색합니다.

    AI 용어, 파이썬 기초, 펫보험 정보를 검색할 수 있습니다.
    source_type 파라미터로 특정 카테고리만 검색 가능합니다.
    """

    # 필터 설정
    if source_type == "all":
        filter_dict = None
    else:
        filter_dict = {"source_type": source_type}

    # retriever객체 생성
    retriever = vector_store.as_retriever(
        search_type="similarity",
        search_kwargs={
            "k": k,
            "filter": filter_dict
        }
    )
```

```
# 검색 실행
results = retriever.invoke(query)

if not results:
    return "검색 결과가 없습니다."
return results
```

7. RAG 에이전트 만들기

- 검색 Tool 만들기

- 에이전트 생성

- 생성한 도구를 사용할 수 있는 에이전트 정의
 - LLM에 RAG 도구를 연결하고, 에이전트에게 ‘어떻게 행동할지’에 대한 명확한 지침이 담긴 시스템 프롬프트를 부여

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

# LLM 설정
model = ChatOpenAI(model="gpt-5-nano", temperature=0)

tools = [
    search_docs
]

system_prompt="""
당신은 AI 기술, 파이썬 프로그래밍, 펫보험에 대해 정보를 제공하는 전문가입니다.
... ( 생략 )...
"""

# RAG Agent 생성
rag_agent = create_agent(
    model=model,
    tools=tools,
    system_prompt=system_prompt
)
```

7. RAG 에이전트 만들기

- 검색 Tool 만들기

- RAG 에이전트 실행

- 에이전트가 실제로 사용자 질문을 분석하고, search_docs 도구를 올바른 인자(source_type)와 함께 호출하는지 스트리밍을 통해 관찰
 - 도구 호출 필요성 판단(추론:reasoning) -> 도구 호출(행동:action) -> 결과 확인(관찰:observation) 과정을 거치고, 정보가 충분하지 않다면 반복
 - 최종 응답을 생성할 준비가 되었다면(완료 결정:finish) 답변을 생성하여 반환
 - 각 과정이 잘 드러날 수 있도록 조건문으로 구분하여 출력

7. RAG 에이전트 만들기

- 검색 Tool 만들기

- RAG 에이전트 실행

```
query = "펫보험의 자기부담금은 어떻게 되나요?"
messages = [{
    "role": "user",
    "content": query
}]

print(f"사용자 질문: {query}")

for chunk in rag_agent.stream({
    "messages": messages
}, stream_mode="values"):

    latest_msg = chunk["messages"][-1]

    if latest_msg.__class__.__name__ == "AIMessage":
        if latest_msg.content:
            print(f"Agent 생각:\n{latest_msg.content[:150]}...\n")

        if latest_msg.tool_calls:
            for tc in latest_msg.tool_calls:
                print(f"도구 호출: {tc['name']}")
                print(f"    입력: {tc['args']}\n")

    elif latest_msg.__class__.__name__ == "ToolMessage":
        print(f"도구 결과:")
        print(f"    {latest_msg.content[:200]}...\n")

print("\n최종 응답:")
print(chunk["messages"][-1].content)
```

7. RAG 에이전트 만들기

● 검색 Tool 만들기

• 에이전트 실행과 도구 호출 관찰

사용자 질문: 펫보험의 자기부담금은 어떻게 되나요?

도구 호출: search_docs

입력: {'query': '펫보험 자기부담금', 'source_type': 'insurance', 'k': 3}

도구 결과:

[Document(id='9f4faec9-ac0d-4e3f-8f31-43a5d6b89cac', metadata={'source_type': 'insurance', 'category': 'Pet Insurance', 'total_pages': 28, 'page_label': '20', 'creationdate': '', 'moddate': '2024-10-1...

도구 호출: search_docs

입력: {'query': '펫보험 자기부담금', 'source_type': 'insurance', 'k': 5}

도구 결과:

[Document(id='9f4faec9-ac0d-4e3f-8f31-43a5d6b89cac', metadata={'total_pages': 28, 'page': 19, 'creator': 'PyPDF', 'source_type': 'insurance', 'moddate': '2024-10-11T00:26:06+00:00', 'source': 'data/me...

- 시스템 프롬프트 중, “4. 찾은 문서의 정보가 부족하면, k 값을 늘려 더 많은 문서를 검색하세요”가 잘 수행되는지 확인하기 위해 의도적으로 에이전트를 여러 번 호출해 봄
- 그 결과, 처음 search_docs를 호출할 때는 k 값을 3으로 설정하여 호출하고는, 문서의 정보가 만족스럽지 않았는지 한 번 더 호출하는 모습을 볼 수 있음
- 두 번째 호출에서는 k 값을 5로 설정하고 있음

7. RAG 에이전트 만들기

- 검색 Tool 만들기

- 에이전트 실행과 도구 호출 관찰

Agent 생각:

간단히 정리하면, 펫보험의 자기부담금은 보험금 지급 전 피보험자(고객)가 먼저 부담하는 금액이나 비율을 말합니다. 담보마다 다르고, 보험사와 상품에 따라 다르게 적용됩니다.

주요 포인트

- 기본 개념: 자기부담금 차감 후 지급. 즉, 보상 대상 의료비에서 일정 금액/...

- 두 번째 호출이 끝난 후에는 충분히 정보가 모였다고 판단했는지, 문서의 내용을 확인

7. RAG 에이전트 만들기

● 검색 Tool 만들기

• 에이전트 실행과 도구 호출 관찰

최종 응답:

간단히 정리하면, 펫보험의 자기부담금은 보험금 지급 전 피보험자(고객)가 먼저 부담하는 금액이나 비율을 말합니다. 담보마다 다르고, 보험사와 상품에 따라 다르게 적용됩니다.

주요 포인트

- 기본 개념: 자기부담금 차감 후 지급. 즉, 보상 대상 의료비에서 일정 금액/비율을 먼저 빼고 남은 금액만 보험금으로 지급됩니다.
- 적용 방식의 예시
 - 고정 금액(일당 또는 사건당): 통원 치료 시 매 방문/일마다 일정 금액을 차감하는 방식.
 - 고정 금액의 예시(메리츠화재의 한 상품 예시): 갱신형 펫퍼민트 Puppy&Family보험2404에서
 - 통원의료비: 1일당 1만원/2만원/3만원
 - 입원의료비: 1일당 1만원/2만원/3만원
 - 배상책임(사고당): 3만원
- 비례보상 여부: 일부 상품은 "일정 금액 또는 일정 비율의 자기부담금"으로 차감되기도 하며, 실손형 상품의 경우 타 보험의 가입 여부에 따라 비례보상으로 보험금을 지급받을 수 있습니다. 이는 보험계약의 조항에 따라 달라집니다.
- 주의점: 정확한 금액은 가입하신 보험상품의 담보표(보험증권의 자기부담금 표)와 상품코드에 따라 다릅니다. 또한 담보별로 적용 방식이 다를 수 있습니다.

... (생략) ...

추가로 원하시면 제가 관련 문서를 더 찾아서 다양한 보험사의 자기부담금 구성을 비교해 드리겠습니다.

- 최종 답변을 생성하여 반환
- 에이전트는 도구의 반환 결과를 바로 최종 답변으로 사용하지 않음. 모인 정보를 토대로 답변을 생성
- 그 결과, PDF 파일의 내용을 사람이 읽기 쉽게 정리하는 것뿐만 아니라 참고 사항도 추가하여 답변한 것을 확인할 수 있음
- 이제 에이전트는 학습하지 않은 새로운 문맥과 지식(사내 데이터 등)에 대해서도 신뢰할 수 있는 답변을 생성할 수 있게 됨